

교과목 3 : 초거대언어모델(LLM)

단원 5 : 파인 튜닝

DAY4. LLM 파인 튜닝 모델의 성능 평가 방법

목 차

1. 강화 학습과 피드백 루프		3
2. RLHF 와 DPO		16
3. DPO Training Data 준비와 실습		29

1. 강화 학습과 피드백 루프

파인 튜닝(Fine-tuning)에서 강화학습(Reinforcement Learning, RL)과 피드백 루프(Feedback Loop)는 모델이 사람의 의도에 맞는 답변을 하도록 지속적으로 개선하는 핵심 과정입니다.

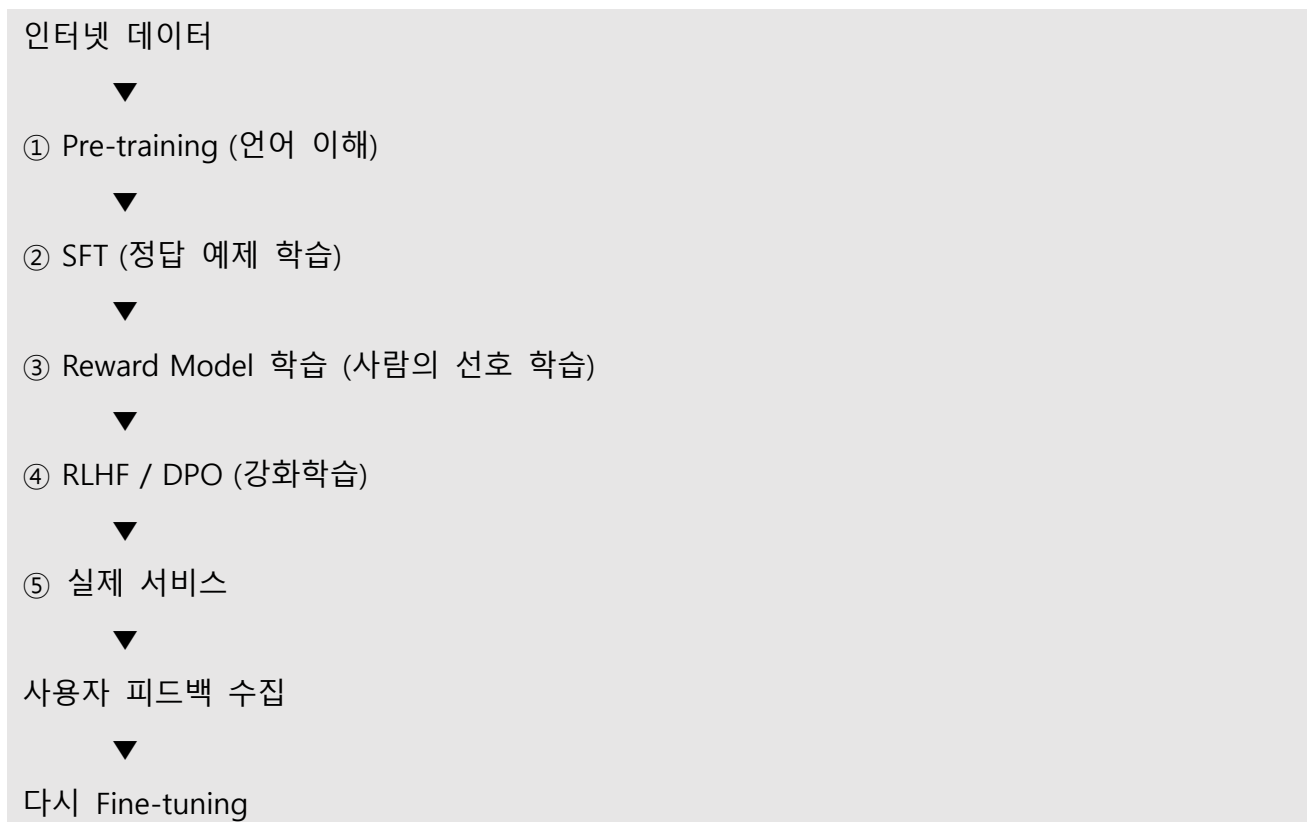
초기의 LLM 은 대부분 사전학습(Pre-training)과 지도학습(SFT, Supervised Fine-Tuning)만으로도 사용할 수 있지만, 실제 서비스에서는 이것만으로는 부족합니다.

왜냐하면

- 여러 답이 가능한 질문이 많고,
- 사람마다 좋은 답이라고 생각하는 기준이 다르며,
- 안전성, 친절함, 정확성 등을 동시에 만족해야 하기 때문입니다.

그래서 사람의 평가를 학습하는 강화학습 단계가 추가됩니다.

전체 학습 과정



이 마지막 과정이 **Feedback Loop** 입니다.

1. 강화학습(Reinforcement Learning)이란?

강화학습은

행동(Action)을 하면 보상(Reward)을 받고, 보상을 최대화하도록 학습하는 방법입니다.

일반적인 머신러닝은

입력
↓
정답(Label)
↓
오차 계산
↓
학습

입니다.

반면 강화학습은

현재 상태(State)
↓
행동(Action)
↓
보상(Reward)
↓
더 좋은 행동 선택

입니다.

예시

사용자가 질문합니다.

파이썬이 무엇인가요?

모델은

답변 A

프로그래밍 언어입니다.

답변 B

파이썬은 배우기 쉽고 데이터 분석, AI, 웹 개발 등에 사용하는 대표적인 프로그래밍 언어입니다.

사람은

B가 더 좋다.

라고 평가합니다.

이 평가가

$\text{Reward} = +1$

이 됩니다.

반대로 A는

$\text{Reward} = 0$

또는

$\text{Reward} = -1$

이 됩니다.

모델은 높은 Reward를 받는 방향으로 조금씩 수정됩니다.

강화학습의 핵심 요소

요소	의미
State	현재 질문
Action	모델이 생성한 답변
Reward	사람이 준 점수
Policy	답변 생성 방법
Environment	사용자와 서비스 환경

즉

질문

↓

답변

↓

평가

↓

좋은 답변 방향으로 수정

입니다.

Fine-tuning 에서 강화학습이 필요한 이유

SFT 는

정답 하나

만 학습합니다.

하지만 실제 질문은

좋은 답이 여러 개

존재합니다.

예를 들어

AI 란?

정답은

인공지능입니다.

도 맞고

사람처럼 학습하는 기술입니다.

도 맞습니다.

그런데 사람은

두 번째가 더 좋다.

라고 생각할 수 있습니다.

이 차이를 배우는 것이 강화학습입니다.

RLHF (Reinforcement Learning from Human Feedback)

현재 가장 유명한 방식입니다.

순서는

SFT 완료

↓

모델이 여러 답 생성

↓

사람이 순위 평가

↓

Reward Model 생성

↓

PPO 강화학습

↓

새 모델 생성

예

질문

한국의 수도는?

모델

답 1

서울입니다.

답 2

서울이며 대한민국의 정치, 경제 중심지입니다.

사람은

답 2

를 선택합니다.

이 데이터를 모아 Reward Model 을 만듭니다.

Reward Model

Reward Model 은 사람 대신 점수를 주는 AI 입니다.

입력

질문

답변

출력

87 점

예를 들어

질문

↓

답변

↓

Reward Model

↓

92 점

이 점수를 이용하여 강화학습합니다.

PPO (Proximal Policy Optimization : 근접 정책 최적화)

RLHF 에서 가장 많이 사용되었습니다.

용어 의미

- **Proximal** : 기존 정책에서 너무 멀리 벗어나지 않는(가까운, 근접한) 것을 의미
- **Policy** : 정책 (에이전트가 상태에서 어떤 행동을 선택할지 결정하는 함수)
- **Optimization** : 최적화

즉, "기존 정책에서 너무 크게 변하지 않도록 제한하면서 정책을 최적화하는 강화학습 알고리즘" 이라는 의미입니다.

PPO 는 정책을 한 번에 크게 변경하지 않고, 작은 범위에서 안정적으로 업데이트하도록 설계되었습니다. 이를 통해 학습이 불안정해지는 문제를 줄이고, 구현도 비교적 단순하면서 높은 성능을 제공하기 때문에 현재 강화학습에서 가장 널리 사용되는 정책 경사(Policy Gradient) 계열 알고리즘 중 하나입니다.

동작은

```
모델 생성
↓
Reward 계산
↓
Policy 수정
↓
다시 생성
↓
Reward 증가
```

입니다.

PPO 의 특징

- 안정적
- 성능 우수
- OpenAI GPT 초기 모델 사용
- 계산량이 매우 큼

DPO (Direct Preference Optimization : 직접 선호도 최적화)

용어 의미

- **Direct** : 보상모델(Reward Model)을 거치지 않고 직접을 의미
- **Preference** : 사람이 더 좋은 답변을 선택한 정보, 선호도를 의미
- **Optimization** : 최적화

즉, "사람이 선호 데이터를 직접 이용하여 언어 모델을 최적화하는 학습 방법"이라는 의미입니다.

PPO 는 정책을 한 번에 크게 변경하지 않고, 작은 범위에서 안정적으로 업데이트하도록 설계되었습니다. 이를 통해 학습이 불안정해지는 문제를 줄이고, 구현도 비교적 단순하면서 높은 성능을 제공하기 때문에 현재 강화학습에서 가장 널리 사용되는 정책 경사(Policy Gradient) 계열 알고리즘 중 하나입니다.

최근에는 PPO 대신 DPO 가 많이 사용됩니다.

이유는 PPO 는

Reward Model 필요

+

강화학습 필요

입니다.

반면

DPO 는

좋은 답

나쁜 답

만 있으면 됩니다.

즉

질문

↓

좋은 답

↓

나쁜 답
↓
직접 학습

입니다.

그래서

- 구현이 쉽고
- 비용이 낮고
- 성능도 우수합니다.

현재 많은 오픈소스 LLM(Qwen, Llama, Mistral 등)의 선호도 학습에서 DPO 가 널리 사용됩니다.

피드백 루프(Feedback Loop)

피드백 루프는

서비스에서 수집된 사용자 반응을 다시 학습 데이터로 활용하여 모델을 지속적으로 개선하는 순환 구조입니다.

구조는 다음과 같습니다.

서비스
↓
사용자 질문
↓
LLM 답변
↓
좋아요 / 싫어요
↓
로그 저장
↓
데이터 정제
↓
재학습
↓
새 모델 배포

즉, 모델을 한 번 학습하고 끝내는 것이 아니라,
사용자의 실제 사용 경험을 기반으로 반복적으로 개선합니다.

Feedback Loop 구성 요소

① 사용자 질문 수집

질문
↓
DB 저장

예

질문
답변
시간
사용자

② 사용자 평가

예를 들어

★★★★★

또는



를 저장합니다.

③ 오류 분석

예

환각(Hallucination)
틀린 답
불친절한 답
너무 긴 답

을 분류합니다.

④ 데이터 생성

예

잘못된 답

↓

수정

↓

새 학습 데이터 생성

⑤ 재학습

SFT

↓

DPO

↓

평가

↓

배포

실제 서비스에서의 Feedback Loop

사용자

↓

FastAPI

↓

LLM

↓

답변

↓

사용자 평가

↓

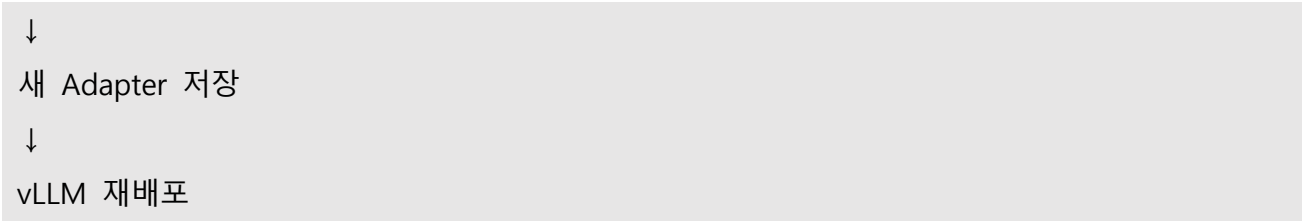
MySQL 저장

↓
관리자 검토
↓
Fine-tuning Dataset 생성
↓
RunPod 학습
↓
새 Adapter 생성
↓
vLLM 배포
↓
서비스 업데이트

이 구조는 사용자 피드백을 기반으로 모델을 점진적으로 개선하는 실무적인 워크플로입니다.

RunPod 기반 실무 예시

PyCharm
↓
FastAPI
↓
vLLM
↓
Fine-tuned Qwen
↓
사용자 질문
↓
답변 생성
↓
Feedback 저장(MySQL)
↓
JSONL 생성
↓
DPO 학습(RunPod)



이와 같이 서비스 → 피드백 → 재학습 → 재배포의 순환이 지속됩니다.

강화학습과 피드백 루프의 차이

구분	강화학습(RL)	피드백 루프(Feedback Loop)
목적	모델의 정책(답변 생성 방식) 최적화	실제 서비스 데이터를 이용한 지속적인 개선
입력	질문, 답변, 보상(선호도)	사용자 질문, 답변, 평가, 로그
핵심 데이터	Reward 또는 선호도 비교 데이터	사용자 행동 및 서비스 로그
수행 시점	모델 학습 단계	모델 운영과 재학습을 반복하는 전체 운영 과정
대표 기법	RLHF, PPO, DPO	데이터 수집→정제→ SFT/DPO →재배포

2. RLHF 와 DPO

LLM 의 학습 과정은 일반적으로 다음과 같이 진행됩니다.

- ① Pre-training (사전학습)
↓
- ② SFT (Supervised Fine-Tuning)
↓
- ③ Preference Learning (RLHF 또는 DPO)
↓
- ④ Evaluation
↓
- ⑤ Deployment

여기서 **RLHF** 와 **DPO** 는 모두 "사람이 선호하는 답변"을 학습시키기 위한 방법입니다.

즉, 정답을 맞추는 것이 아니라 사람이 더 좋아하는 답변을 생성하도록 모델을 학습시키는 단계입니다.

왜 RLHF 와 DPO 가 필요한가?

SFT 에서는 다음과 같이 학습합니다.

질문

파이썬이란?

정답

프로그래밍 언어입니다.

Loss

Cross Entropy Loss

이렇게 학습하면

모델은

정답과 최대한 비슷한 문장

을 생성합니다.

그러나 실제 서비스에서는

좋은 답변

은 하나가 아닙니다.

예를 들어

답변 A

파이썬은 프로그래밍 언어입니다.

답변 B

파이썬은 AI, 웹 개발,
데이터 분석 등에 많이 사용하는
배우기 쉬운 프로그래밍 언어입니다.

둘 다 맞습니다.

하지만 대부분의 사용자는

B

를 선호합니다.

즉 **정답(Label)**이 아니라 **선호도(Preference)**를 학습해야 합니다.

RLHF의 등장 배경

기존 지도학습은

Question

↓

Ground Truth

↓

Loss

입니다.

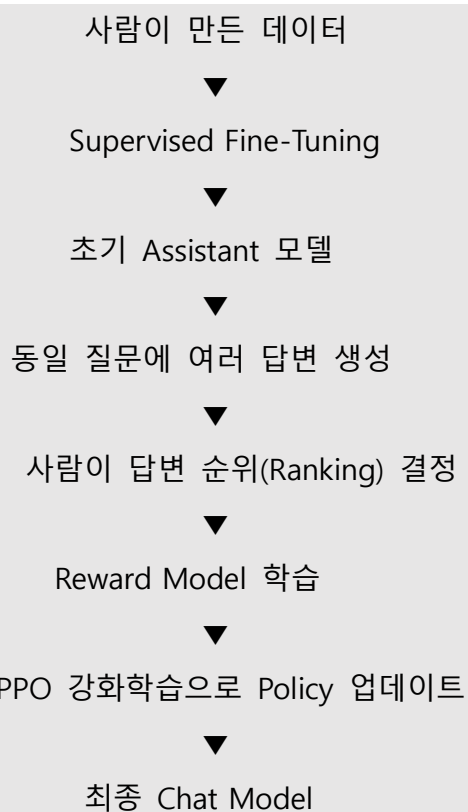
하지만 ChatGPT 같은 대화형 AI 는

질문
↓
답변 생성
↓
사람 평가
↓
좋은 답변 방향 수정

이 필요했습니다.

그래서 등장한 것이 **RLHF**(Reinforcement Learning from Human Feedback) 입니다.

RLHF 전체 구조



RLHF 는 크게 **4 단계**로 이루어집니다.

1 단계 : SFT

먼저 일반적인 Fine-tuning 을 합니다.

Dataset

Question

↓

Answer

예

```
{  
  "instruction": "AI 란?",  
  "output": "인공지능입니다."  
}
```

Loss 는

Cross Entropy

입니다.

이 단계에서는 모델이

기본적인 답변

을 생성할 수 있게 됩니다.

2 단계 : Preference Dataset 생성

이제 같은 질문에 모델이 여러 개 답변을 생성합니다.

예를 들어

질문

AI 란?

답변 A

인공지능입니다.

답변 B

AI 는 사람처럼 학습하고 추론하는 컴퓨터 기술입니다.

사람이

B

를 선택합니다.

이렇게

Chosen
Rejected

쌍이 만들어집니다.

예

```
{  
  "prompt": "AI 란?",  
  "chosen": "사람처럼 학습하는 기술입니다.",  
  "rejected": "컴퓨터 프로그램입니다."  
}
```

이 데이터가 매우 중요합니다.

3 단계 : Reward Model

Reward Model 은 사람 대신 점수를 매기는 모델입니다.

입력

Question
+
Answer

출력

Reward = 8.7

예

질문

↓

답변

↓

Reward Model

↓

92 점

Reward Model 은 사람의 선호를 Regression 문제처럼 학습합니다.

Loss 는 대표적으로

Bradley-Terry Loss

또는

Pairwise Ranking Loss

를 사용합니다.

Reward Model 수식

Reward Function

$R(x,y)$

x : 질문

y : 답변

출력

Reward Score

예

$R(\text{question}, \text{answer})=9.8$

Reward Model 의 역할

예를 들어

답변

서울입니다.

점수

72

답변

서울이며 대한민국의 수도입니다.

점수

95

Reward Model 은 항상 높은 점수를 주는 방향으로 Assistant 를 학습시킵니다.

4 단계 : PPO

이제 진짜 강화학습입니다.

PPO (Proximal Policy Optimization) 구조

Question

↓

Assistant 생성

↓

Reward 계산

↓

Policy Update

↓

새로운 Assistant

PPO 가 필요한 이유

일반 Gradient Descent 는 한 번에 너무 많이 업데이트될 수 있습니다.

PPO 는 Policy 가 갑자기 변하지 않도록 조금씩 수정합니다.

즉 안정성을 보장합니다.

PPO Objective

핵심 식

$$L = \text{PPO} = \min(rA, \text{clip}(r, 1 - \epsilon, 1 + \epsilon)A)$$

의미는 너무 크게 업데이트하지 말자. 입니다.

RLHF 의 장점

- 사람의 선호를 잘 반영
- 안전성 증가
- 대화 품질 향상
- 거절 응답 개선
- 유해 답변 감소

RLHF 의 단점

- Reward Model 필요
- 사람 평가 비용 큼
- PPO 구현 어려움
- GPU 비용 큼
- 학습 속도 느림

DPO 의 등장

Meta 논문 이후 가장 많이 사용하는 방식입니다.

논문 : Direct Preference Optimization

핵심 아이디어는 Reward Model 없이 직접 Preference 를 학습하자.

DPO 구조

```
SFT Model
↓
Chosen
Rejected
↓
Preference Loss
↓
Fine-tuning
↓
Final Model
```

Reward Model 이 없습니다.
PPO 도 없습니다.

DPO 핵심 아이디어

우리가 이미

```
좋은 답
나쁜 답
```

을 알고 있다면 굳이 Reward Model 을 만들 필요가 없습니다.
바로 좋은 답의 확률을 높이면 됩니다.

DPO 데이터

예

```
{
  "prompt": "AI 란?",
  "chosen": "사람처럼 학습하는 기술입니다.",
  "rejected": "컴퓨터 프로그램입니다."
}
```

이 데이터만 있으면 됩니다.

DPO 수학적 개념

기존 RLHF

Preference
↓
Reward Model
↓
Reward
↓
PPO
DPO
Preference
↓
Loss
↓
Gradient

바로 끝입니다.

DPO Loss

핵심 아이디어

Chosen 확률 ↑
Rejected 확률 ↓

입니다.

대표 형태는 로지스틱 손실을 이용해 **선호된 응답의 로그 확률이 비선호 응답보다 높아지도록** 최적화합니다. 이때 현재 정책과 기준(reference) 정책의 차이를 함께 고려하여 모델이 과도하게 변하는 것을 방지합니다.

직관적으로는

Chosen Probability >> Rejected Probability
--

가 되도록 학습합니다.

DPO 가 빠른 이유

- Reward Model 없음
- PPO 없음
- Rollout 없음
- Environment 없음

즉 일반 Fine-tuning 처럼 학습합니다.

RLHF 와 DPO 비교

항목	RLHF	DPO
SFT 필요	O	O
Reward Model	O	X
PPO	O	X
강화학습	O	직접적인 RL 사용
Preference Dataset	O	O
구현 난이도	높음	낮음
GPU 사용량	많음	적음
학습 속도	느림	빠름
유지보수	어려움	쉬움
최근 사용 빈도	감소 추세	증가 추세

실무에서의 선택

RLHF 를 사용하는 경우

- 매우 큰 상용 모델
- 복잡한 온라인 보상 설계
- 실제 상호작용을 통한 정책 최적화가 필요한 경우
- 연구 목적의 강화학습

대표 사례

- 초기 ChatGPT
- GPT-3.5 계열의 RLHF 파이프라인
- 일부 Claude 계열 연구

DPO 를 사용하는 경우

- 오픈소스 LLM 파인튜닝
- 기업 내부 LLM
- LoRA/QLoRA 학습
- Hugging Face TRL 기반 학습
- RunPod GPU 환경
- Qwen, Llama, Mistral, Gemma 계열 모델의 선호도 튜닝

대표 사례

- Qwen 계열 DPO 튜닝
- Llama 3 기반 선호도 학습
- Mistral 계열 모델
- Gemma 계열 모델

RunPod 기반 DPO 학습 흐름

PyCharm



FastAPI 서비스



사용자 질문/답변 저장



Chosen / Rejected 데이터 생성



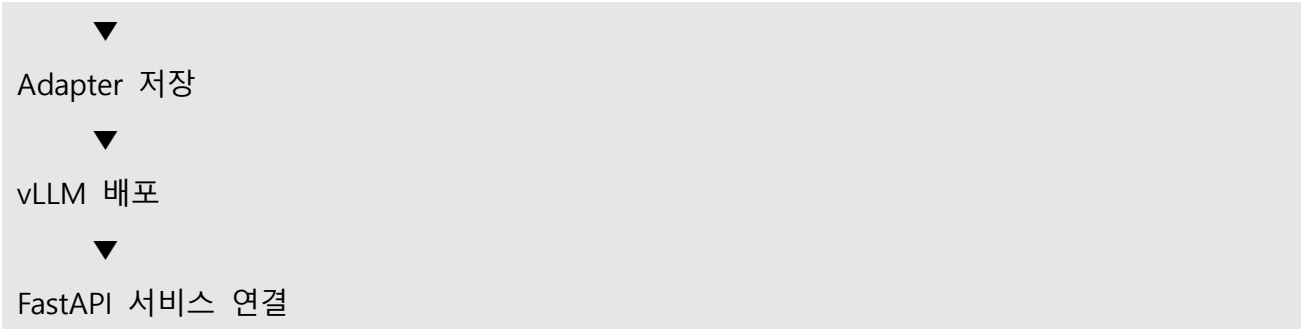
JSONL 데이터셋 구축



RunPod GPU



Qwen + LoRA + DPO 학습



이 방식은 현재 교육, 연구, 기업의 오픈소스 LLM 커스터마이징에서 널리 사용되는 실무적인 워크플로입니다.

핵심 정리

구분	RLHF	DPO
핵심 목표	사람의 선호를 보상 기반 강화학습으로 반영	사람의 선호를 직접 손실 함수로 최적화
핵심 구성	SFT → Reward Model → PPO	SFT → Preference Dataset → DPO Loss
데이터	질문, 답변, 선호도	질문, Chosen, Rejected
장점	복잡한 보상 설계와 정책 최적화 가능	구현이 단순하고 학습 효율이 높음
단점	구현과 계산 비용이 큼	보상 함수를 명시적으로 설계하기 어려움
현재 활용	대규모 연구 및 일부 상용 모델	오픈소스 LLM 파인튜닝의 사실상 표준

3. DPO Training Data 준비와 실습

포함 기능



Hugging Face TRL은 DPOTrainer를 제공하며, prompt, chosen, rejected 형태의 선호도 데이터로 DPO 학습을 수행할 수 있습니다. PEFT의 LoRA를 결합하면 전체 모델 파라미터를 모두 학습하지 않고 일부 Adapter 파라미터만 학습할 수 있어 GPU 메모리와 저장 공간을 줄일 수 있습니다. ([Hugging Face](#))

프로젝트 구조

```
dpo_training_runpod_project/
├── app/
│   ├── __init__.py
│   └── main.py
│
├── data/
│   ├── raw/
│   │   └── preferences.jsonl
│   └── processed/
│       └── preferences_clean.jsonl
│
├── scripts/
│   ├── 01_prepare_data.py
│   ├── 02_train_dpo.py
│   ├── 03_evaluate.py
│   └── run_runpod.sh
│
├── src/
│   ├── __init__.py
│   ├── config.py
│   └── data_utils.py
│
├── tests/
│   └── test_data_utils.py
│
├── outputs/
├── .env.example
├── .gitignore
├── requirements.txt
└── README.md
```

DPO 데이터 형식

```
{  
  "prompt": "GPU 메모리가 부족할 때 LLM 파인 튜닝 방법을 제안하세요.",  
  "chosen": "4비트 양자화와 LoRA를 결합한 QLoRA를 사용하고, 배치 크기를 줄인 뒤  
gradient accumulation과 gradient checkpointing을 적용하는 방법이 효과적입니다.",  
  "rejected": "GPU 메모리가 부족해도 전체 파라미터를 FP32로 학습하면 됩니다."  
}
```

각 필드의 역할은 다음과 같습니다.

필드	의미
prompt	모델에 전달할 질문 또는 지시문
chosen	사람이 선호하는 우수한 답변
rejected	상대적으로 품질이 낮은 답변

프로젝트에서는 Hugging Face Datasets의 JSON 로더로 로컬 JSONL을 불러오고 학습·평가 데이터로 분할합니다. Hugging Face Datasets는 로컬 JSON과 JSONL 파일 로딩을 지원합니다. ([Hugging Face](#))

PyCharm에서 데이터 검증

압축 해제 후 PyCharm 터미널에서 실행합니다.

```
cd C:\LLM_workspace\ddpo_training_runpod_project  
  
python -m venv .venv  
.\venv\Scripts\activate  
  
python -m pip install --upgrade pip  
pip install -r requirements.txt  
  
copy .env.example .env
```

데이터 검증:

```
python scriptsW01_prepare_data.py
```

정상 결과:

```
[검증 성공] 6개 레코드를 저장했습니다:  
dataWprocessedWpreferences_clean.jsonl
```

테스트 실행:

```
pytest -q
```

RunPod에서 실행

프로젝트를 다음 경로에 배치하는 것을 권장합니다.

```
/workspace/dpo_training_runpod_project
```

RunPod SSH 연결은 SSH 공개키를 RunPod 계정에 등록한 뒤 Pod의 SSH 접속 정보를 PyCharm Remote Development에 설정하는 방식으로 사용할 수 있습니다. RunPod는 공개키 등록 방식의 SSH 연결을 지원하고, PyCharm은 SSH 기반 Remote Development 연결을 제공합니다. ([Runpod Documentation](#))

RunPod 터미널:

```
cd /workspace/dpo_training_runpod_project  
cp .env.example .env  
nano .env
```

.env 예시:

```
HF_TOKEN=hf_XXXXXXXXXXXXXXXXXXXXXXXXXXXXX  
BASE_MODEL=Qwen/Qwen2.5-0.5B-Instruct  
ADAPTER_PATH=outputs/dpo_adapter  
HOST=0.0.0.  
PORT=8000
```


전체 자동 실행:

```
bash scripts/run_runpod.sh
```

이 스크립트는 다음 작업을 순차적으로 수행합니다.

- 가상환경 생성
- 패키지 설치
- 데이터 검증
- DPO 학습
- 성능 평가

단계별 실행

1. 데이터 준비

```
python scripts/01_prepare_data.py
```

2. DPO 학습

```
python scripts/02_train_dpo.py ₩  
--epochs 1 ₩  
--batch-size 1 ₩  
--gradient-accumulation 8
```

기본 학습 설정:

Base Model:
Qwen/Qwen2.5-0.5B-Instruct

Quantization:
4bit NF4

Fine-tuning:
LoRA / QLoRA

LoRA Rank:

16

DPO Beta:

0.1

Maximum Length:

1024

Maximum Prompt Length:

512

LoRA는 대규모 모델의 주요 가중치를 고정하고 작은 저차원 행렬만 학습하는 PEFT 방식이므로 전체 파라미터 파인 튜닝보다 메모리 사용량을 크게 줄일 수 있습니다. ([Hugging Face](#))

GPU 없이 구조만 확인할 때:

```
python scripts/02_train_dpo.py --no-4bit
```

CPU 학습은 실행 시간이 매우 길 수 있으므로 실제 학습은 RunPod CUDA GPU 환경이 적합합니다.

3. 성능 평가

```
python scripts/03_evaluate.py
```

평가 결과 예시:

기준 모델 선호도 정확도: 0.5000

DPO 모델 선호도 정확도: 0.8333

정확도 변화: +0.3333

평가 스크립트는 각 데이터에 대해 다음 값을 비교합니다.

```
log P(chosen | prompt)
log P(rejected | prompt)
```

다음 조건을 만족하면 해당 데이터에서 선호도를 올바르게 학습한 것으로 계산합니다.

```
log P(chosen | prompt) > log P(rejected | prompt)
```

다만 제공된 샘플 6개는 **코드 실행 확인용 데이터**입니다. 실제 모델 성능 판단에는 학습에 사용하지 않은 별도의 홀드아웃 테스트 데이터가 필요합니다.

FastAPI 추론 서비스

학습 완료 후 실행합니다.

```
uvicorn app.main:app ♻️
  --host 0.0.0.0 ♻️
  --port 8000
```

서버 상태 확인:

```
curl http://127.0.0.1:8000/health
```

응답:

```
{
  "status": "ok",
  "model_loaded": true,
  "cuda_available": true
}
```

답변 생성:

```
curl -X POST http://127.0.0.1:8000/generate ₩
-H "Content-Type: application/json" ₩
-d '{
  "prompt": "GPU 메모리가 부족할 때 파인 튜닝 방법을 설명하세요.",
  "max_new_tokens": 200,
  "temperature": 0.7
}'
```

실무 데이터 준비 시 주의점

DPO 성능은 모델보다 데이터 쌍의 품질에 크게 좌우됩니다.

chosen은 단순히 길고 친절한 답이 아니라 실제 서비스 정책과 정확성 기준을 만족해야 합니다. rejected는 터무니없이 잘못된 답보다는 현재 모델이 실제로 생성할 가능성이 있는 열등한 답이 학습에 더 유용합니다.

예를 들어 다음 데이터는 좋지 않습니다.

```
{
  "prompt": "SQL 인젝션 예방법은?",
  "chosen": "파라미터 바인딩과 최소 권한을 적용합니다.",
  "rejected": "바나나는 노란색입니다."
}
```

두 답변의 차이가 너무 명확하여 모델이 SQL 보안 품질이 아니라 단순한 주제 일치만 학습할 수 있습니다.

더 적절한 rejected는 다음과 같습니다.

```
{
  "prompt": "SQL 인젝션 예방법은?",
  "chosen": "파라미터 바인딩 또는 ORM을 사용하고 최소 권한과 입력 검증을 함께 적용합니다.",
  "rejected": "사용자 입력에서 작은따옴표만 제거하면 완전히 예방됩니다."
}
```

이 프로젝트는 TRL 0.24 계열 API를 기준으로 버전을 고정했습니다. TRL은 발전 속도가 빠른 라이브러리이므로 임의로 최신 버전만 설치하기보다 프로젝트의 requirements.txt 버전을 유지하는 것이 재현성에 유리합니다. ([Hugging Face](#))